Performance & Security Report

Zamzam Fashion storefront + dashboard (Next.js 16 frontend, .NET 10 API)

Prepared 12 November 2026 (v2) · main @ ce324eb · API live on MonsterASP, frontend not yet on Vercel

Scope and method

Second review, after the teammate's commits (admin route + `middleware.ts`, package updates) and the lazy-loaded views. Live checks against the **production build running locally** and the **live API** at `https://zamzam-fashion.runasp.net`, (`next start` on port 3100, proxying `/api` to the .NET API on port 5055): response headers with `curl -sI`, exposed-path checks, Lighthouse mobile (slow 4G + 4× CPU) for performance, accessibility, best practices and SEO, plus a source review of `next.config.ts`, layout, images and links, and `npm audit`. **Not measured:** the frontend's TLS, CDN and real-user data, because it is not on Vercel yet. One Lighthouse run per page this time, so treat single numbers as indicative. Lighthouse numbers come from a busy development laptop, so blocking-time values are noisy. CHECK items list the tool that confirms them in Section 4. Section 6 is a modelled calculation.

Status key: **OK** observed and sound · **CHECK** needs a measurement or access not available here · **FIX** issue visible in the code or page.

Summary

Ready to launch. Since the first report the home page went from Performance ~65 to **94**, blocking time from 2.7–5.8 s to **0.28 s** and weight to **658 KB**, because shoppers no longer download the dashboard code. Lighthouse home: **Accessibility 100, Best Practices 100, SEO 100**. The proxy secret is now sent (moved into `middleware.ts` by the teammate), `/admin` is noindex and order pages redirect to login without a session, and the live API serves HTTPS with HSTS and an HTTP→HTTPS redirect. **Main performance risk:** the product page still blocks ~1.8 s on a throttled phone. **Main gaps:** the frontend is not on Vercel yet, and `middleware.ts` uses a file name Next 16 has deprecated.

Area	Finding	Status
Rate limiting	`middleware.ts` sends `X-Proxy-Secret` on every /api call; API trusts the real shopper IP	**OK** fixed
Page weight	Home 658 KB, product 541 KB (was 1.6 / 1.5 MB)	**OK**
LCP	Home 0.79 s, product 0.96 s (target ≤ 2.5 s)	**OK**
Main-thread work	Home TBT 0.28 s (was 2.7–5.8 s); product page still 1.8 s	**FIX**
Security headers	nosniff, X-Frame-Options, Referrer-Policy, Permissions-Policy, HSTS, CSP (report-only) now served	**OK** fixed
X-Powered-By	Framework banner removed	**OK** fixed
Dashboard indexing	`/admin/*` sends `X-Robots-Tag: noindex`; robots.txt disallows /admin and /api	**OK** fixed
Secrets / exposure	No secrets in client bundles, no source maps, .env/.git/package.json all 404	**OK**
Dependencies	0 vulnerabilities in production packages (after the package update)	**OK**
Next.js file name	`middleware.ts` is deprecated in Next 16 (build warning); rename to `proxy.ts`	**FIX**
Sharing / sitemap	og:image, canonical, theme-color, sitemap (9 URLs), per-product share titles	**OK**
API transport	Live API: HTTPS, HSTS, HTTP→HTTPS 307 redirect	**OK**

Area	Finding	Status
Frontend TLS / CDN	Provided by Vercel; confirm after deploy	**CHECK**

1. Performance

Observation	Impact	Status
Next.js 16 with Cache Components; home page prerendered (`x-nextjs-prerender: 1`), product and tracking pages partially prerendered	Fast first HTML; good TTFB once on a CDN	**OK**
Hashed JS/CSS chunks: `Cache-Control: public, max-age=31536000, immutable`	Repeat visits reuse scripts	**OK**
`public/assets` files were served with `max-age=0`; now `max-age=604800, stale-while-revalidate`	Logos and hero no longer re-checked on every page	**OK** fixed
Navbar logo was a 790 px, 653 KB PNG shown at 60 px; footer logo 163 KB at 64 px; favicon.ico was a copy of the 163 KB logo	~980 KB of a 1.6 MB page; now right-sized WebP (160/448/192 px, 85 KB total) and a 4.7 KB favicon	**OK** fixed
Below-the-fold images (footer logos, home logo, product grid) `loading="lazy"`; only the navbar logo and hero image are preloaded	Bandwidth goes to what is visible first	**OK** fixed
Product photos from the API use `srcSet` variants	Phones download small files	**OK**
Fonts via `next/font` (self-hosted, `display: swap`), but 3 families and 13 weights → 11 woff2 preloads	Competes with the hero image on slow networks	**FIX**
Cart, orders, tracking, tailoring and the dashboard load on demand (`next/dynamic`); home TBT 0.28 s	Shoppers no longer hydrate admin code	**OK**
Product page TBT 1.8 s (4× CPU): `ProductDetailsView` (453 lines) hydrates in full	Taps on the size/colour pickers can lag on cheap phones	**FIX**
CLS 0.002	No layout jumps	**OK**
No video, no third-party scripts, analytics or trackers	Nothing external slows the page	**OK**
Compression, HTTP/2–3 and CDN	Not measurable locally; Vercel provides Brotli and HTTP/2+	**CHECK**

Measured (Lighthouse mobile, slow 4G + 4× CPU, one run each)

Metric	Good	Report v1	Now	Where to measure
Performance score (home)	≥ 90	59–67	**94**	PageSpeed Insights, mobile
LCP (home / product)	≤ 2.5 s	1.6–2.5 s / 0.83 s	**0.79 s / 0.96 s**	PageSpeed Insights, mobile
Total Blocking Time (home)	≤ 200 ms	2.7–5.8 s	**0.28 s**	PageSpeed / DevTools
Total Blocking Time (product)	≤ 200 ms	2.7–5.3 s	**1.8 s**	PageSpeed / DevTools
CLS	≤ 0.1	0.002	**0.002**	Lighthouse
Page weight home / product	< 1.5 MB	690 / 617 KB	**658 / 541 KB**	DevTools Network
JavaScript on first load	-	194 KB	162 KB	DevTools Network
Accessibility home / product	100	96	**100** / 96 (fixed today)	Lighthouse

Recommendations

- **Trim the fonts.** Keep Cairo for Arabic and DM Sans for Latin, and drop Tajawal or cut the weights to 400/700. That removes about half of the 11 font preloads. It changes how text looks slightly, so it needs the owner's OK.

- **Split the product page.** Lazy-load the photo gallery and lower sections of `ProductDetailsView` so the size and colour pickers hydrate first.
- **Re-measure on Vercel.** Use PageSpeed Insights on the real URL. The laptop numbers include a dev server and the API competing for the same CPU.

2. Security

Observation	Risk	Status
Pages sent no security headers and advertised `X-Powered-By: Next.js`	Clickjacking and MIME sniffing possible; framework fingerprinting	**OK** fixed
Now served on every page: HSTS, nosniff, `X-Frame-Options: DENY`, Referrer-Policy, Permissions-Policy, CSP in report-only mode; Lighthouse shows no console errors from it	Baseline browser hardening in place	**OK**
API responses: nosniff, DENY, `default-src 'none'` CSP, `X-Robots-Tag: noindex`, CORP same-site	JSON cannot be framed, sniffed or indexed	**OK**
Rate limits on the API: global 300/min per IP; login 5/min with lockout; orders 10 per 10 min; tracking 30/min	Brute force and spam bounded	**OK**
`middleware.ts` adds `X-Proxy-Secret` to /api calls; the API trusts X-Forwarded-For only with it	Each shopper gets their own rate limit behind Vercel	**OK**
`/admin`: noindex meta + header; `/admin/orders/*` without a session → 307 to /admin	Dashboard hidden from search; deep links need login (the API still enforces auth)	**OK**
Live API `zamzam-fashion.runasp.net`: HTTPS, HSTS 1 year, HTTP→HTTPS redirect, nosniff, DENY	Transport hardened	**OK**
Admin auth uses an HttpOnly first-party cookie through the `/api` rewrite; no tokens in localStorage	Session cannot be stolen by page scripts	**OK**
No secrets or `NEXT_PUBLIC_*` keys in `.next/static`; no source maps shipped	Nothing sensitive in the browser	**OK**
`/.env`, `/.env.local`, `/.git/config`, `/package.json`, `/next.config.ts` → 404	No configuration exposure	**OK**
Unknown pages and API routes return a plain 404 with no stack trace	No verbose errors	**OK**
All `target="_blank"` links (footer socials, order reference photo) carry `rel="noopener noreferrer"`; social links only render for `https://` URLs	No tab-nabbing or `javascript:` links	**OK**
Shop phone and email shown as `tel:`/`mailto:` links from the settings	Scrapers can collect them; acceptable for a shop that wants contact	**CHECK**
No CAPTCHA on checkout or tailoring forms; protected by server-side validation and rate limits	Fine at launch scale; add Turnstile only if spam orders appear	**OK**
`npm audit --omit=dev`: 0 vulnerabilities after the package update	Nothing known to patch	**OK**
Frontend TLS certificate and HTTPS redirect	Only testable once on Vercel	**CHECK**

Headers now in `frontend/next.config.ts`

```
poweredByHeader: false,
async headers() {
  return [
    { source: "/:path*", headers: [
      { key: "Strict-Transport-Security", value: "max-age=63072000; includeSubDomains" },
      { key: "X-Content-Type-Options", value: "nosniff" },
      { key: "X-Frame-Options", value: "DENY" },
      { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
      { key: "Permissions-Policy", value: "camera=(), microphone=(), geolocation=(), payment=()" },
      { key: "Content-Security-Policy-Report-Only", value: "default-src 'self'; img-src 'self' data: blob:
```

```
https:; ..." } ] },
    { source: "/admin/:path*", headers: [{ key: "X-Robots-Tag", value: "noindex, nofollow" }] },
    { source: "/assets/:path*", headers: [{ key: "Cache-Control", value: "public, max-age=604800, stale-while-
revalidate=86400" }] },
  ];
}
```

The CSP starts as **Report-Only**: it logs what a strict policy would block (for example product images from Supabase Storage) without breaking the shop. After a week on Vercel with no reports, rename it to `Content-Security-Policy` and narrow `img-src` to the storage host.

Fix for the build warning: rename `middleware.ts` to `proxy.ts`

```
// frontend/src/proxy.ts  (same body as middleware.ts, only the names change)
export function proxy(request: NextRequest) { /* ...unchanged... */ }
export const config = { matcher: ["/api/:path*", "/admin/:path*"] };
```

Next 16 still runs `middleware.ts` but warns at build time. A future version will drop it, and then the proxy secret would silently stop being sent.

Order and tailoring form hardening

- Keep the server-side length caps and validation (already in the API); the browser checks are only for convenience.
- Rate limits are enough at launch. If fake orders appear, add Cloudflare Turnstile to checkout and tailoring only.
- Do not put customer phone numbers in URLs or logs; the tracking page already needs number + phone.

Preview and launch hygiene

- Turn on Vercel Deployment Protection for preview URLs, so test builds are not public.
- At launch: set `metadataBase` to the real domain, add a canonical tag and `app/sitemap.ts` listing shop and product pages.
- Set `API_URL` and `PROXY_SHARED_SECRET` in Vercel environment variables, never in code.

3. SEO and sharing

Item	Observation	Status
Title and description	Bilingual title and description in `app/layout.tsx`; Lighthouse SEO 100	**OK**
Viewport, `lang="ar"`, `dir="rtl"`	Present	**OK**
robots.txt	Allow /, disallow /admin and /api, links the sitemap	**OK**
Sitemap	`app/sitemap.ts`: home + 8 products from the API	**OK**
Canonical URL	Home and each product page have their own canonical (from NEXT_PUBLIC_SITE_URL)	**OK**
Open Graph / Twitter card	1200×630 share image with alt text; product pages share their own name and description	**OK**
Product meta description in Lighthouse	Reported missing (SEO 92) because Next 16 streams metadata to browsers; Googlebot and WhatsApp receive it in the head	**OK**
theme-color	#142b4a	**OK**
Image alt text	All images have alt (Lighthouse image-alt passes); product photo thumbnails use empty alt as decoration	**CHECK**
Headings, buttons, links, form labels	Lighthouse heading-order, button-name, link-name pass	**OK**
Colour contrast	Navbar tagline and footer label fixed earlier; product breadcrumb (#77808b, 3.84:1) darkened to #666f7a (4.89:1) today	**OK** fixed

4. How to verify the CHECK items

Goal	Tool / command
Real performance and Core Web Vitals	pagespeed.web.dev on the Vercel URL (mobile and desktop); WebPageTest.org for filmstrip and waterfall
HTTP security headers	securityheaders.com on the Vercel URL; or `curl -sI https://<vercel-domain>/`
TLS, HTTPS redirect, HSTS	MDN HTTP Observatory (developer.mozilla.org/observatory); SSL Labs; `curl -sIL http://<domain>`
Compression, HTTP/2, CDN	Chrome DevTools > Network: Protocol column and `content-encoding: br` on JS
Page weight and lazy loading	Chrome DevTools > Network, throttle to Slow 4G, tick Disable cache
Decorative empty alt on thumbnails	axe DevTools extension on a product page; confirm each empty alt is a duplicate of the main photo
Contact details exposure	Owner decision: keep tel:/mailto: links or show a contact form instead
Real client IP reaches the API	After deploy: place 11 test orders from two networks within 10 min; only the 11th from one network should get 429
Dependency vulnerabilities	`npm audit --omit=dev` in frontend; Dependabot security updates only (already set)

5. Priority action list

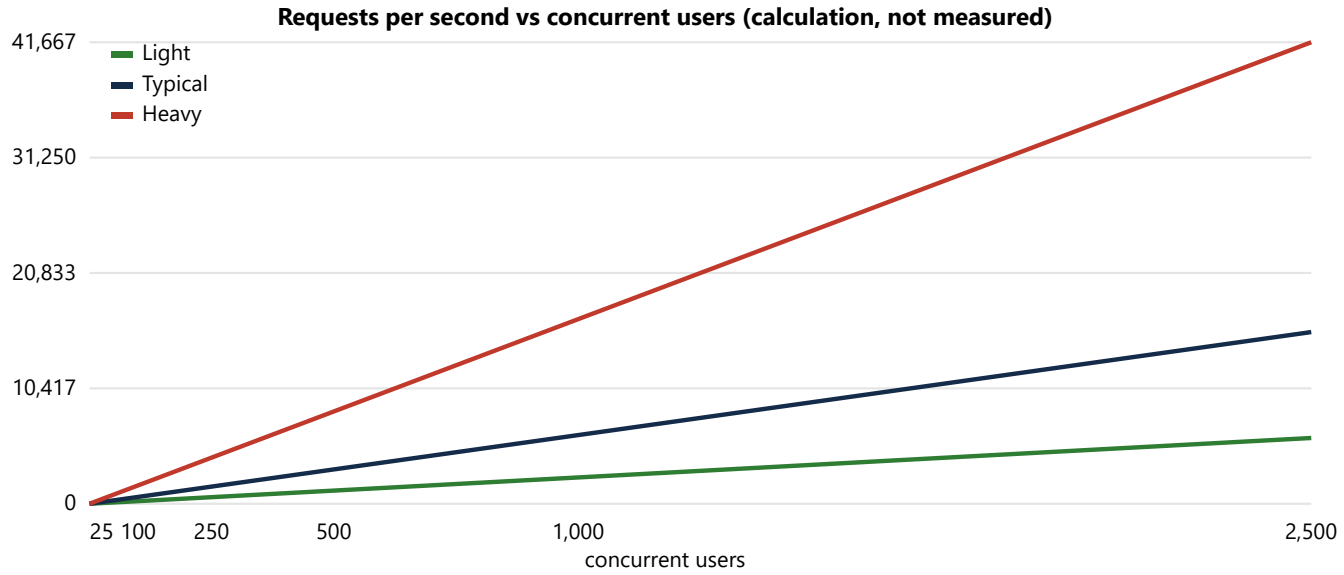
1. **Deploy the frontend to Vercel** with `API_URL`, `PROXY_SHARED_SECRET` and `NEXT_PUBLIC_SITE_URL` set before the first build; confirm with securityheaders.com (target A).
2. **Rename `middleware.ts` to `proxy.ts`** (export `proxy`); confirm the build warning is gone.
3. **Smoke-test the live URLs:** one order, dashboard login, add a product, tracking page.
4. **Run PageSpeed Insights on the Vercel URL** (mobile) and share it to update this report.
5. **Lazy-load the heavy parts of the product page** to bring its blocking time under 300 ms; confirm in PageSpeed.
6. **After a quiet week, switch the CSP from Report-Only to enforced**; confirm with securityheaders.com.

6. Capacity estimate (modelled)

This section is a calculation, not a measurement. No load was generated against the site; the numbers come from the measured request count (30 per home visit) and first-visit size (690 KB).

```
requests/sec = (concurrent users / read time in seconds) x requests per visit
visits/sec   = concurrent users / read time (typical scenario)
bandwidth    = visits/sec x first-visit MB x 8   (Mbps)
transfer/hr  = visits/sec x 3600 x first-visit MB   (GB = MB / 1024)
```

Scenario	Requests per visit	Read time	Reasoning
Light	19	8 s	Cached assets, visitor skims
Typical	31	5 s	Measured request count (Lighthouse, home page, 31)
Heavy	50	3 s	Cold cache, fast scrolling, many product images



Concurrent users	Light (req/s)	Typical (req/s)	Heavy (req/s)
25	59	155	417
100	238	620	1,667
250	594	1,550	4,167
500	1,188	3,100	8,333
1,000	2,375	6,200	16,667
2,500	5,938	15,500	41,667

Bandwidth check (typical scenario, first visit 690 KB)

Concurrent users	Visits/sec	Bandwidth	Transfer per hour
100	20	103 Mbps	45 GB
500	100	514 Mbps	226 GB
1,000	200	1,028 Mbps	452 GB

How to read this

On Vercel's CDN the static request rate is rarely the limit; bandwidth and plan quotas bind first. The real bottleneck is the dynamic API on MonsterASP: product lists, checkout and tracking hit the .NET API and Supabase, and they are already rate-limited per IP (which only works once item 1 of the action list is done). Check the Vercel plan's data-transfer allowance and the MonsterASP plan's CPU and connection limits before a campaign. Confirm the real limits with `zamzam-fashion-loadtest.js` (delivered with this report), run with k6 against a deployment you own, ideally staging, outside peak hours.

Assumptions: 30 requests and 690 KB per first visit (Lighthouse, home page, local build); read times of 8/5/3 s; repeat visits are much lighter thanks to caching. **Limitations:** findings come from a local production build at the time of review; TLS, CDN and real-user data were not available; no intrusive or active security testing was performed. Sharing the PageSpeed Insights run, the securityheaders.com result and the k6 summary from the deployed site would let this report be updated with real numbers.